SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

SE2020 - Web Technologies and Mobile

Ms. Dilani Lunugalage

BSc (Hons) in Information Technology

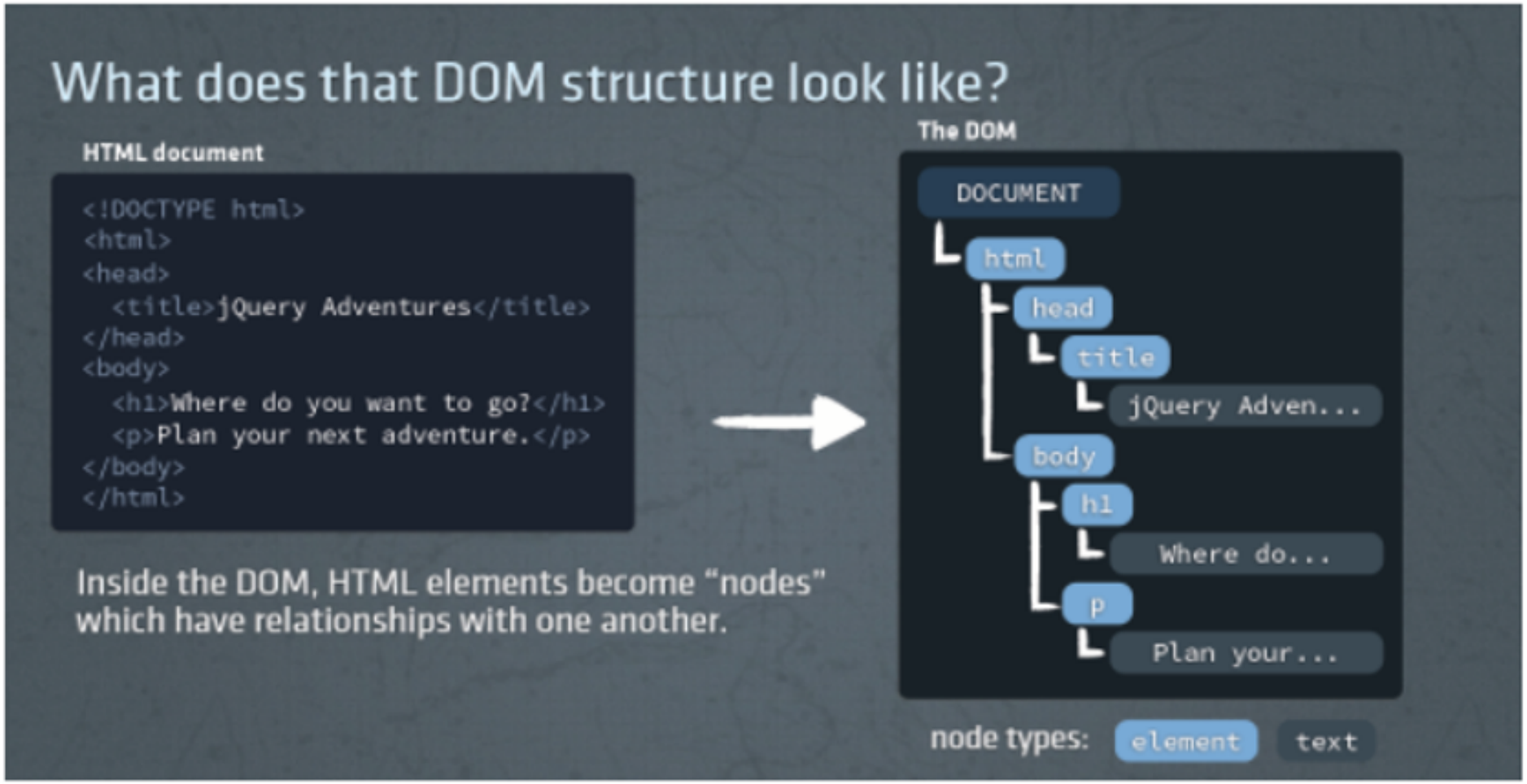
Lecture 04

HTML DOM

Introduction to the HTML DOM (Document Object Model)

- We will only cover aspects of the DOM that we need to read/change form element values/settings, document content and document style attributes, so we will not cover the DOM in full
- The Document Object Model is a platform- and language-neutral interface that will allow programs and scripts to dynamically access and update the content, structure and style of documents.
- The document can be further processed and the results of that processing can be incorporated back into the presented page. This is an overview of DOM-related materials here at W3C and around the web. (W3C)
- The HTML DOM model is constructed as a tree of Objects, created by the browser, so we can quickly find HTML elements using JavaScript

DOM


What does that DOM structure look like?
HTML document
<!DOCTYPE html>
<html>
<head>
<title>jQuery Adventures</title>
</head>
<body>
<h1>Where do you want to go?</h1>
Plan your next adventure.
</body>
</html>
The DOM
DOCUMENT
html
head
title
jQuery Adven...
body
h1
Where do...
p
Plan your...
Inside the DOM, HTML elements become "nodes" which have relationships with one another.
node types:
element
text

JavaScript and the DOM

JavaScript can be used to:

- change HTML elements in the page
- change HTML attributes in the page
- change the CSS styles in the page
- react to the events in the page

Finding HTML Elements

- JavaScript is often used to manipulate HTML elements.
- To do so, you have to find the elements first. You can do this by:
- finding HTML elements by id
- finding HTML elements by tag name
- finding HTML elements by class name

DOM - Methods

Document Object Methods

Method	Description	W3C
close()	Closes the output stream previously opened with document.open()	Yes
getElementsByName()	Accesses all elements with a specified name	Yes
getElementById()	Accesses the element with the specified id	Yes
getElementsByClassName()	Accesses all elements with a specified class name	Yes
getElementsByTagName()	Accesses all elements with a specified tag name	Yes
open()	Opens an output stream to collect the output from document.write() or document.writeln()	Yes
write()	Writes HTML expressions or JavaScript code to a document	Yes
writeln()	Same as write(), but adds a newline character after each statement	Yes

DOM - Methods

Finding an HTML Element by `id`

```
let x = document.getElementById( "id1" );
```

Finding HTML Elements by tag name

```
let x = document.getElementsByTagName( "p" );
```

Finding HTML Elements by `class`

```
let x = document.getElementsByClassName( "myClass" );
```

Setting Style Properties to HTML Elements

Syntax

```
document.getElementById( "id1" ).style.property = new style;
```

Example

```
<p id="id2">Hello World!</p>

<script>
document.getElementById( "id2" ).style.color = "blue";
</script>
```

Example

```
<p id="id3">Hello World!</p>

<script>
let myParagraph = document.getElementById( "id3" );
myParagraph.style.color = "blue";
myParagraph.style.fontSize = "30";
</script>
```

If you are going to get the same element more than once, it is more efficient to store the requested element in a variable and use that variable instead.

Document Object Properties

Property	Description	W3C
anchors	Returns a collection of all the anchors in the document	Yes
applets	Returns a collection of all the applets in the document	Yes
body	Returns the body element of the document	Yes
cookie	Returns all name/value pairs of cookies in the document	Yes
documentMode	Returns the mode used by the browser to render the document	No
domain	Returns the domain name of the server that loaded the document	Yes
forms	Returns a collection of all the forms in the document	Yes
images	Returns a collection of all the images in the document	Yes
lastModified	Returns the date and time the document was last modified	No
links	Returns a collection of all the links in the document	Yes
readyState	Returns the (loading) status of the document	No
referrer	Returns the URL of the document that loaded the current document	Yes
title	Sets or returns the title of the document	Yes
URL	Returns the full URL of the document	Yes

Object Properties Can Be Other Objects

- Objects can be partly made up of, or contain, other objects.
- Here is part of the browser's heirarchy of ready made JavaScript objects:

```
window
   |
   . parent              //the "parent" window or frame
   |
   . frames              //an array of frames in the window/frame
   |
   . location            //the URL of the document in the window/frame
   |
   . document            //the document in the window or frame
        |
        . anchors        //an array reflecting all the anchors in a document
        |
        . links          //an array reflecting all the links in a document
        |
        . forms          //an array reflecting all the forms in a document
           |
           . elements    //an array reflecting all the elements in a form
```

Dot Notation

- Statements such as this are common in JavaScript programming.

```
return = awindow.document.aform.elements[ 2 ].value;
```

- awindow refers by name to a window, document is the document in that window, aform is the name of a form in the document, elements[2] refers to the third element in the form and value will return the value of that element.

Changing HTML Style with the HTML DOM Style Object

Text properties

Property	Description	W3C
color	Sets or returns the color of the text	Yes
direction	Sets or returns the text direction	Yes
font	Sets or returns font-style, font-variant, font-weight, font-size, line-height, and font-family in one declaration	Yes
fontFamily	Sets or returns the font face for text	Yes
fontSize	Sets or returns the font size of the text	Yes
fontSizeAdjust	Sets or returns the font aspect value	Yes
fontStyle	Sets or returns whether the style of the font is normal, italic or oblique	Yes
fontVariant	Sets or returns whether the font should be displayed in small capital letters	Yes
fontWeight	Sets or returns the boldness of the font	Yes
letterSpacing	Sets or returns the space between characters in a text	Yes
lineHeight	Sets or returns the distance between lines in a text	Yes

Background properties

Property	Description	W3C
background	Sets or returns all the background properties in one declaration	Yes
backgroundAttachment	Sets or returns whether a background-image is fixed or scrolls with the page	Yes
backgroundColor	Sets or returns the background-color of an element	Yes
backgroundImage	Sets or returns the background-image for an element	Yes
backgroundPosition	Sets or returns the starting position of a background-image	Yes
backgroundRepeat	Sets or returns how to repeat (tile) a background-image	Yes

HTML DOM Events

- HTML DOM events allow JavaScript to register different event handlers on elements in an HTML document.
- Events are normally used in combination with functions, and the function will not be executed before the event occurs (such as when a user clicks a button).
- Some common uses of events
  - When a user clicks the mouse
  - When a web page has loaded
  - When an image has been loaded
  - When the mouse moves over an element
  - When an input field is changed
  - When an HTML form is submitted
  - When a user strokes a key

Mouse Events

Property	Description
onclick	The event occurs when the user clicks on an element
ondblclick	The event occurs when the user double-clicks on an element
onmousedown	The event occurs when a user presses a mouse button over an element
onmousemove	The event occurs when the pointer is moving while it is over an element
onmouseover	The event occurs when the pointer is moved onto an element
onmouseout	The event occurs when a user moves the mouse pointer out of an element
onmouseup	The event occurs when a user releases a mouse button over an element

Key Board Events

Attribute	Description
onkeydown	The event occurs when the user is pressing a key
onkeypress	The event occurs when the user presses a key
onkeyup	The event occurs when the user releases a key

Form Events

Attribute	Description
onblur	The event occurs when a form element loses focus
onchange	The event occurs when the content of a form element, the selection, or the checked state have changed (for <input>, <select>, and <textarea>)
onfocus	The event occurs when an element gets focus (for <label>, <input>, <select>, textarea>, and <button>)
onreset	The event occurs when a form is reset
onselect	The event occurs when a user selects some text (for <input> and <textarea>)
onsubmit	The event occurs when a form is submitted

Object Events

Object Events

Attribute	Description
onabort	The event occurs when an image is stopped from loading before completely loaded (for <object>)
onerror	The event occurs when an image does not load properly (for <object>, <body> and <frameset>)
onload	The event occurs when a document, frameset, or <object> has been loaded
onresize	The event occurs when a document view is resized
onscroll	The event occurs when a document view is scrolled
onunload	The event occurs once a page has unloaded (for <body> and <frameset>)

Reacting to Events

- A JavaScript can be executed when an event occurs, like when a user clicks on an HTML element.
- To execute code when a user clicks on an element, add JavaScript code to an HTML event attribute:

```
onclick = JavaScript
```

Image Replacement

Image Properties

Property	Description
align	Sets or returns the value of the align attribute of an image
alt	Sets or returns the value of the alt attribute of an image
border	Sets or returns the value of the border attribute of an image
complete	Returns whether or not the browser is finished loading an image
height	Sets or returns the value of the height attribute of an image
hspace	Sets or returns the value of the hspace attribute of an image
longDesc	Sets or returns the value of the longdesc attribute of an image
lowsrc	Sets or returns a URL to a low-resolution version of an image
name	Sets or returns the name of an image
src	Sets or returns the value of the src attribute of an image
useMap	Sets or returns the value of the usemap attribute of an image
vspace	Sets or returns the value of the vspace attribute of an image
width	Sets or returns the value of the width attribute of an image

Image Replacement

- Replacing an image already loaded onto a page is done by changing the image object's src attribute to either the URL of another image file or to the src of another image object.
- Here are two code examples:

```
document.images[0].src = "http://www.somehost.co.uk/images/myImg.gif";
```

```
cacheimage = new Image();
cacheimage.src = "http://www.somehost.co.uk/images/myImg.gif";
firstimage = new Image();
firstimage.src = document.images[0].src;
document.images[0].src = cacheimage.src;
.
.
.
document.images[0].src = firstimage.src;
```

A Simple Example

- Changing the URL of the image using the src property causes the new image to be loaded gradually onto the page.
- Replacement is not immediate.

```
<img src="simple0.jpg" width="257" height="75" name="sample" alt="simple image">

<form>
<input type="button" value="simple0.jpg" onclick="document.images[0].src = 'simple0.jpg'">
<input type="button" value="simple1.jpg" onclick="document.images[0].src = 'simple1.jpg'">
</form>
```

Preloading Images into Cache

- Using the image object it is possible to begin loading both images as the page loads.
- Page loading is slowed down as both images are loaded and so the first image may take longer to appear.
- Once they are both loaded image replacement is very fast.

Preloading Images into Cache

```
<script>
cache0 = new Image(257,75)
cache0.src = "cache0.jpg"
cache1 = new Image(257,75)
cache1.src = "cache1.jpg"
</script>

<img
 src   ="cache0.jpg"
 width ="257"
 height="75"
 name  ="sample">

<form>
<input type="button" value="simple0.jpg"
  onclick="document.images[0].src = cache0.src">
<input type="button" value="simple1.jpg"
  onclick="document.images[0].src = cache1.src">
</form>
```

The setTimeout() method

Syntax

```
window.setTimeout( "javascript function", milliseconds );
```

- The window.setTimeout() method can be written without the window prefix.
- The setTimeout() method will wait the specified number of milliseconds, and then execute the specified function.
- The first parameter of setTimeout() should be a function.
- The second parameter indicates how many milliseconds, from now, you want to execute the first parameter.

Example

Wait 3 seconds, then alert "Hello":

```
setTimeout( function() {
    alert( "Hello" )
    }, 3000
);
```

How to Stop the Execution?

The clearTimeout() method is used to stop the execution of the function specified in the setTimout() method.

Syntax

```
window.clearTimeout(timeoutVariable)
```

The window.clearTimout() method can be written without the window prefix. To be able to use the clearTimout() method, you , must use a global variable when creating the timeout method.

```
myVar = setTimeout( "javascript function", milliseconds );
```

Then , if the function has not already been executed, you will be able to stop the execution by calling the clearTimeout() method.

Example

Example

```
let myVar;

function myFunction() {
  myVar = setTimeout( function() {
     alert( "Hello" );
  }, 3000 );
}

function myStopFunction() {
   clearTimeout( myVar );
}
```

The addEventListener() method

- We have used DOM events previously by adding listeners directly in the html element.
- However, in order to totally separate the HTML from the JavaScript code, it is better practice to add the event Listener inside the JavaScript code by using the addEventListener() method.
- This improves separation of concerns, readability and allows you to add the event listeners and effects even if you do not have control over the HTML code.
- The addEventListener() method attaches an event handler to a specified element.

The addEventListener() method

- It will not overwrite existing event handlers on that element, so it is possible to add many event handlers to one element, including more than one identical event handler.
- The other advantage of using the addEventListener() method, is that you get control of the event bubbling (how the event propagates through the DOM nodes).
- You can remove an event listener by using the removeEventListener() method.

The addEventListener() method

Syntax

```
element.addEventListener(event, function, useCapture);
```

Where:

- `element` is the element on which we add the eventListener
- `event` is the event that we are listening for
- `function` is the function that gets called when the event is triggered. This can be a defined function or an annonymous function.
- `useCapture` is a boolean that specifies whether we use the event bubbling (false, also default option) or event capturing (true). This parameter is optional.

Example – Change Background color

```
<div id="turnPurple" style="height:150px; width:150px; border: solid thin black; margin: 10p
</div>
<div id="clearPurple" style="height:150px; width:150px; border: solid thin black; margin: 10
</div>
<script>
document.getElementById( "turnPurple" ).addEventListener( "click", function() {
        document.getElementById( "turnPurple" ).style.backgroundColor = "purple";
});
document.getElementById( "clearPurple" ).addEventListener( "click", function() {
        document.getElementById( "turnPurple" ).style.backgroundColor = "white";
});
</script>
```

Click Me I'll turn purple

Click Me to clear purple

The removeEventListener() method

The removeEventListner() method is used to remove an event listner from an element

Syntax

```
element.removeEventListener("mousemove", myFunction);
```

Thank You!